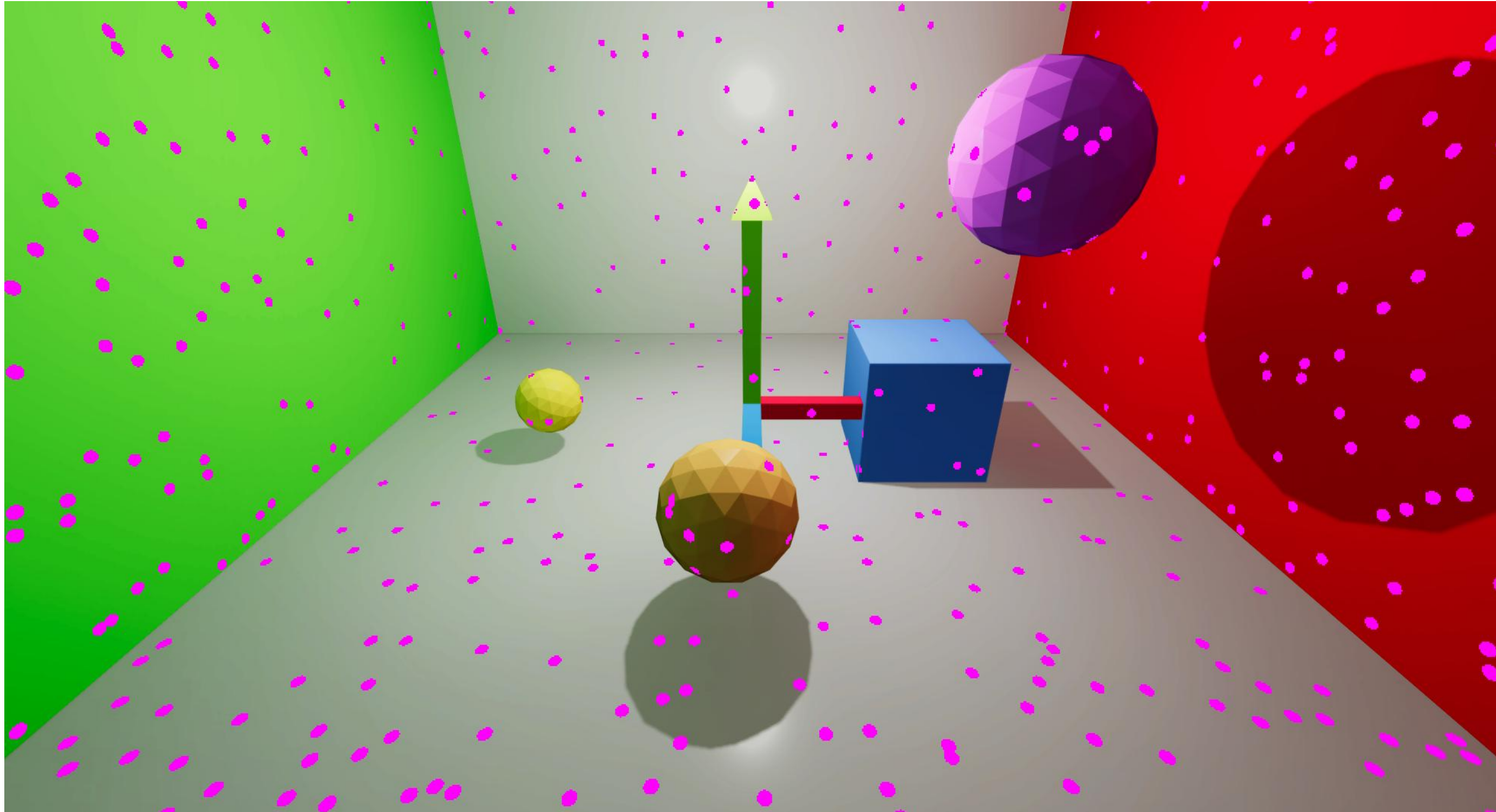


Surfel-based Global Illumination

Surfel-based Global Illumination (Surfel GI)

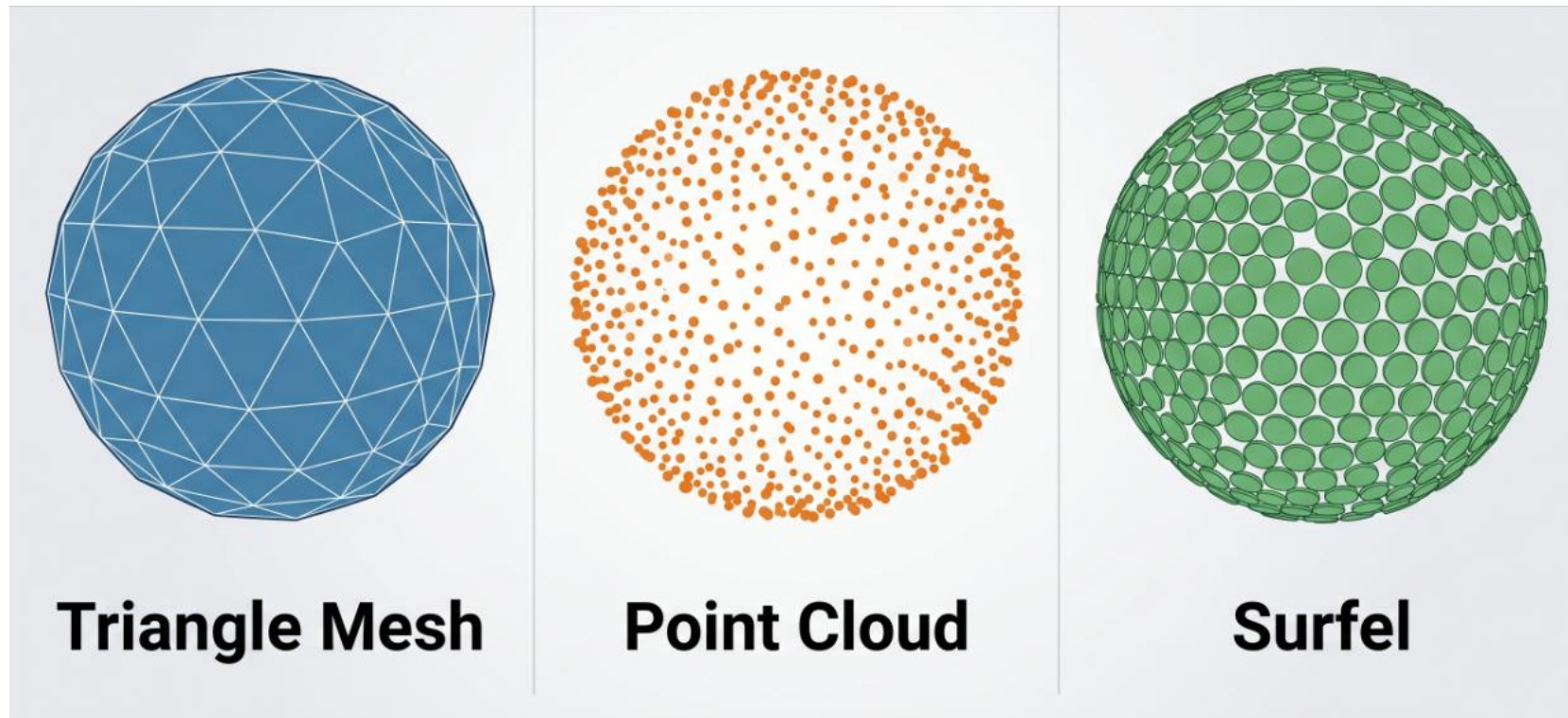


1. Ray tracing from each **surfel** → Capture radiance
2. Store as **Spherical harmonics(SH)** per **surfel**
3. Reuse at Ray tracing from each **surfel** again! → **Recursive** feedback
4. Infinite bounces approximated across frames

Surfel-based Global Illumination (Surfel GI)

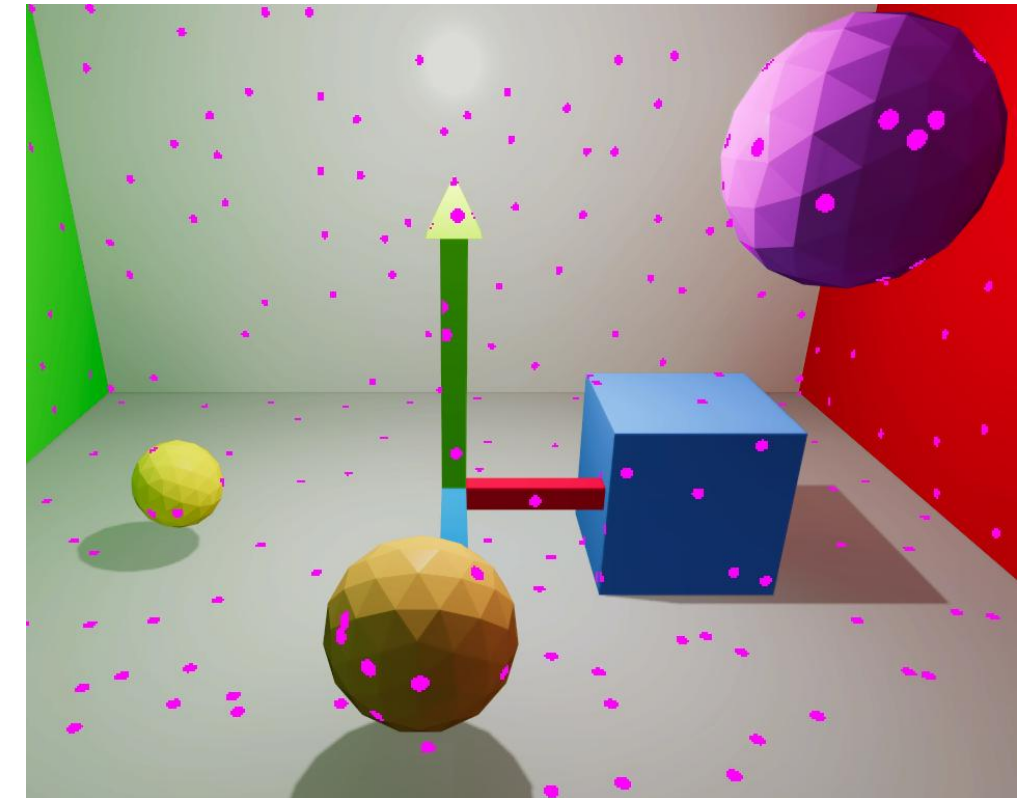
Surfel (Surface Element)

Surfels as Rendering Primitives



≠

Surfels in Surfel GI

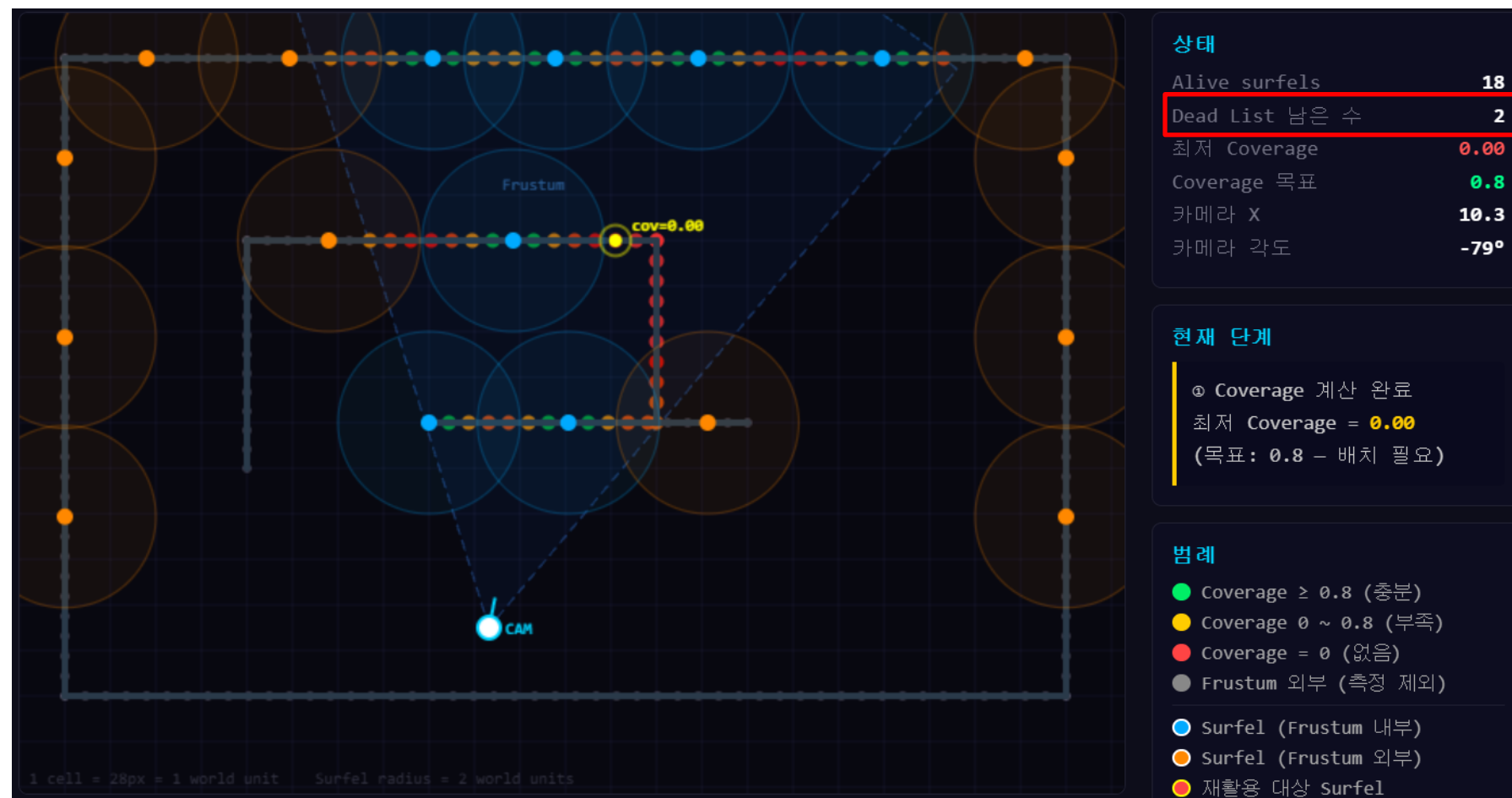


Originally, surfels were used as rendering primitives, similar to triangle meshes or point clouds.
In Surfel GI, surfels are used as surface lighting cache samples.
Their surface-oriented representation is efficient for sparse lighting storage and reuse.

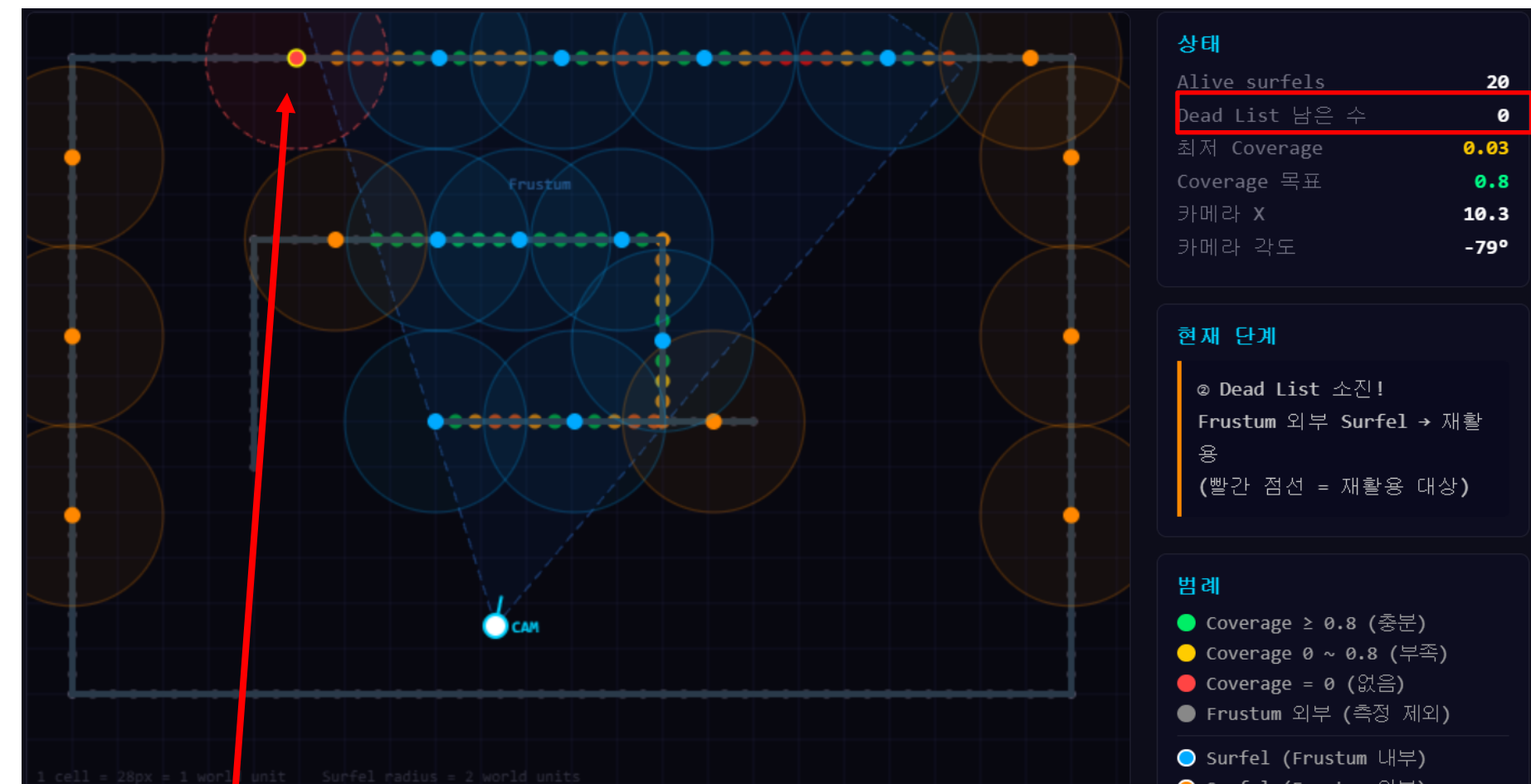
Surfel-based Global Illumination (Surfel GI)

Surfel Placement Mechanism

1. Compute **coverage** for all screen pixels (**number of surfels** within a radius of 2 world units)
2. Try to place a surfel at the pixel with **the lowest coverage**



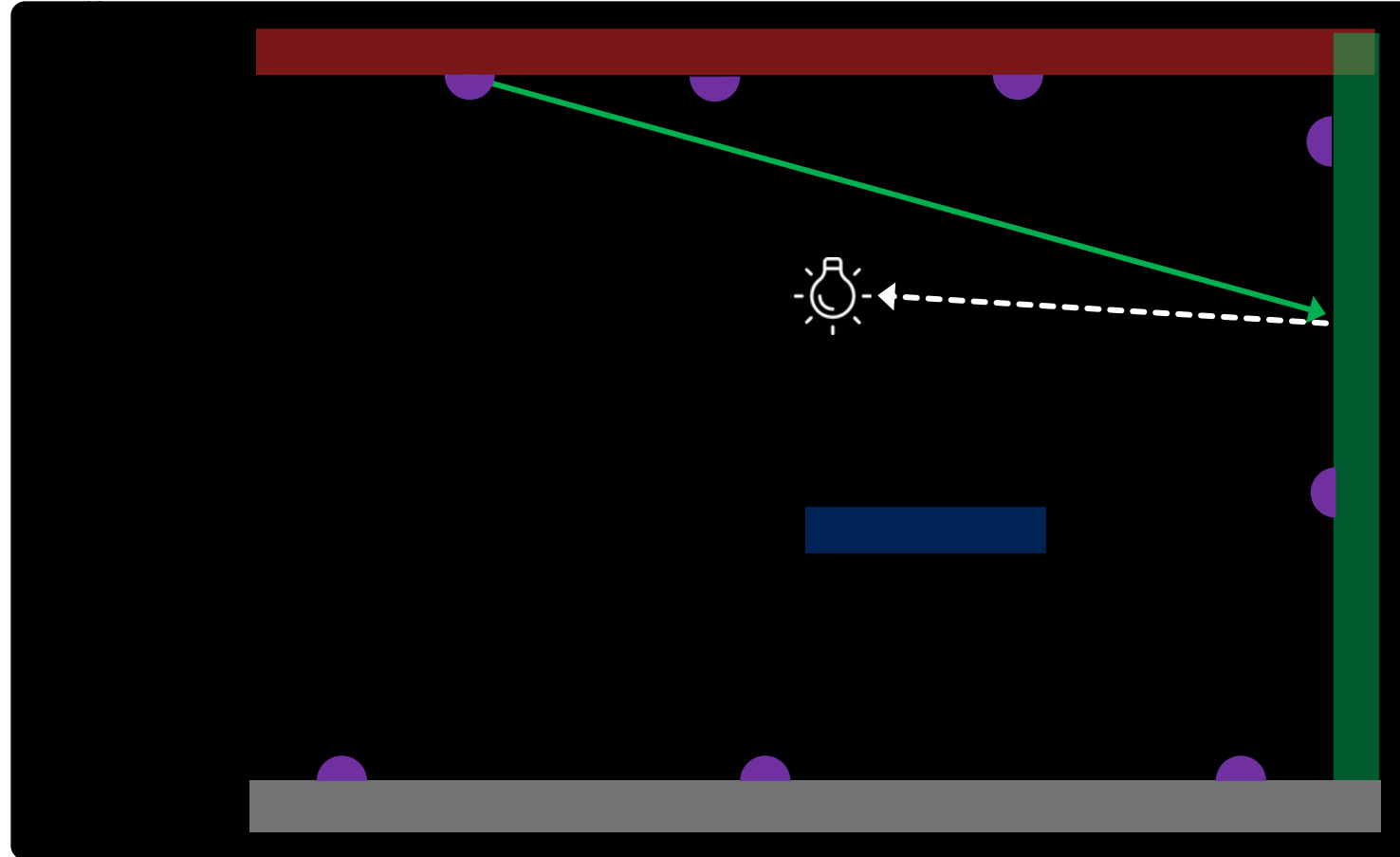
If the dead list has sufficient capacity,
place surfels at the most under-covered pixels



If the dead list is insufficient,
reuse surfels outside the view frustum for placement

Surfel-based Global Illumination (Surfel GI)

direct



Ray firing

- Fires rays from the **surfel position** into the **hemisphere aligned to surface normal N**
- Cosine-weighted hemisphere sampling : directions **closer to N** are sampled with higher probability (importance sampling for Lambertian diffuse)

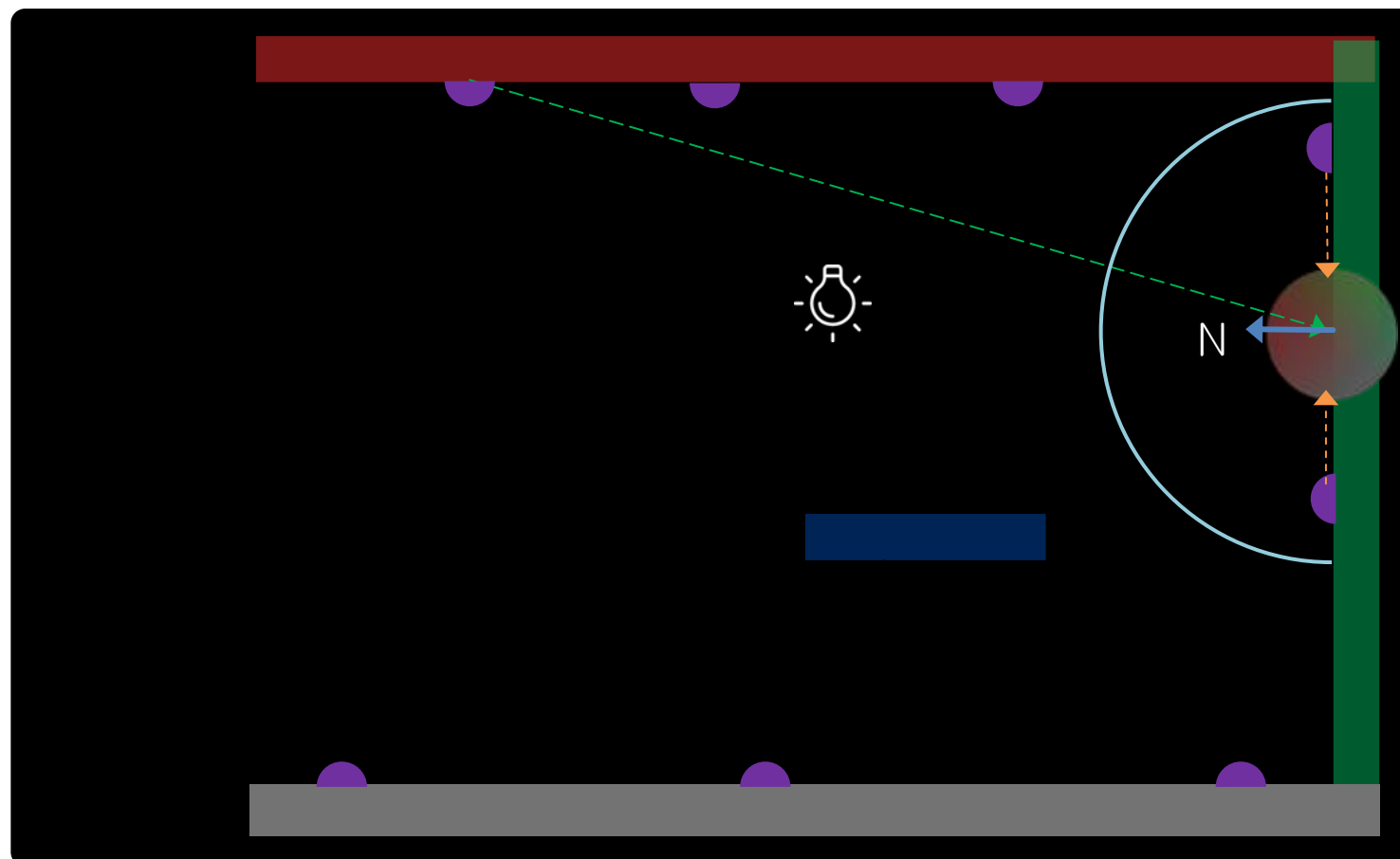
Direct lighting (same as DDGI)

- Randomly sample **one light source**, cast **shadow ray** toward it
- If not occluded: compute reflected radiance at the hit surface

Indirect lighting (infinite bounces)

- **Same approach as DDGI: blend nearby surfel SHs** with contribution weights (weighted blend)
→ **integrate** over surface normal N
- Lookup range: neighboring surfels within the **sampling radius**

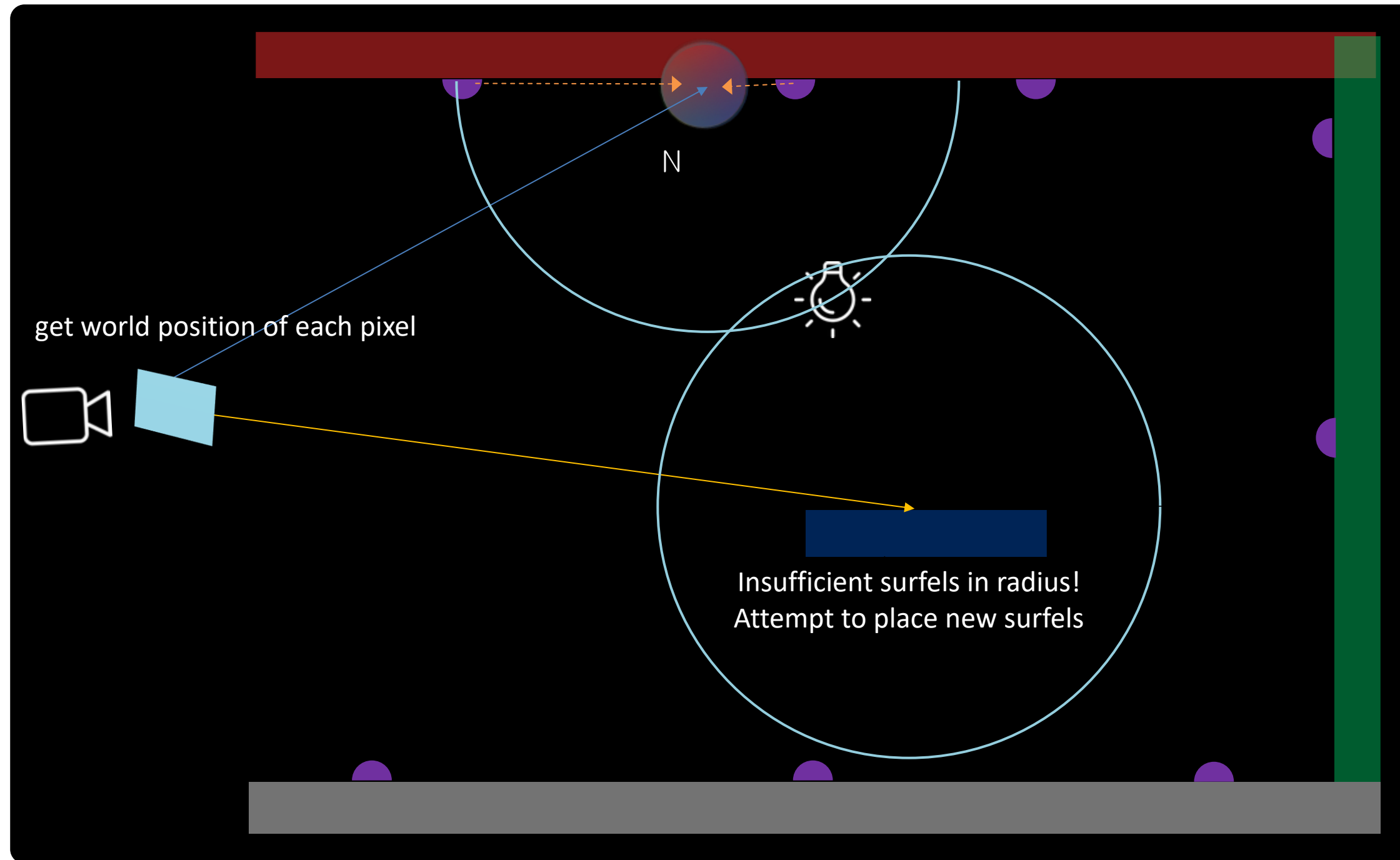
indirect radiance



Uses a pipeline nearly identical to DDGI,
the main difference is how lighting information is stored.

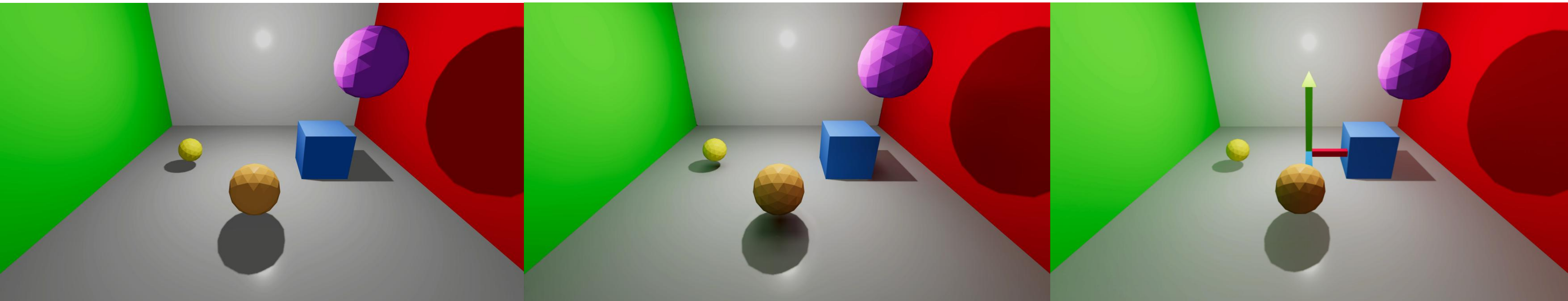
Surfel-based Global Illumination (Surfel GI)

SurfelGI in the Rasterization Pixel Shader



1. The pixel shader reconstructs the **world-space position** of each pixel
2. **Indirect lighting** is sampled from **nearby surfels within the sampling radius** around the **world position**.
3. If there are no surfels, or **insufficient surfels** within the radius around the world position, **attempt to place new surfels**.
4. The sampled indirect lighting is combined with the engine's existing lighting model (Phong, PBR, ...) to produce the final shaded result

Surfel-based Global Illumination (Surfel GI)



original

DDGI

SurfelGI

	DDGI	SurfelGI
Radiance storage	L1 SH over full sphere	L1 SH over hemisphere
Ray sampling direction	Spherical Fibonacci over full sphere	Cosine-weighted hemisphere aligned to surface N
Cache placement	Fixed 3D grid — exists even in empty space	Surface-attached — only where geometry exists
GI accuracy limit	Structures smaller than the probe grid spacing may leak or become inaccurate	Structures smaller than the surfel radius (2 world units) may leak or become inaccurate
Cache management	Simple — fixed grid, no lifecycle	Complex — spawn / accumulate / recycle lifecycle
Specular	X	X